



Laboratorio No.2 Semana #13 – API Secrets y autenticación

Nombre: _____ Carné: _____

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

Parte 1 – Probar autenticación con Postman

Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

1. Crear la solicitud de inicio de sesión

- Abra Postman y cree una nueva colección llamada: MIA – Semana 13.
- Cree una solicitud llamada: Login.
- Seleccione el método POST.
- Utilice el endpoint: `https://dummyjson.com/auth/login`
- En Body seleccione raw y JSON.
Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.
`{ "username": "emilys", "password": "emilyspass" }`
- Ejecute la solicitud y localice en la respuesta el campo `accessToken`.

2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: `https://dummyjson.com/user/me`
- En Authorization seleccione Bearer Token.
- Copie el `accessToken` obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.

3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.
- Registre el código HTTP y explique la diferencia.

4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del `accessToken` y su envío mediante el encabezado `Authorization: Bearer`.



Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada: **AZURE_STORAGE_CONNECTION_STRING**
La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- No publique el archivo .env en GitHub**, ya que contiene la credencial real.

8. Crear el Secret en Github

- Ingresa al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: **AZURE_STORAGE_CONNECTION_STRING**
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.



Mario Maldonado's Workspace

Upgrade

OST POST PUT GET GET GET DEL POST Log

No Environment

MIA - Semana 13 - API > Login

Save Share

POST https://dummyjson.com/auth/login Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "username": "emilys",
3   "password": "emilyspass"
4 }
```

Body 200

{ JSON Preview Visualize

```
1 {
2   "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJlbWlseXMiLCJlbWFWb24uY29tIiwiaXN0IjoiJmZw1hbGUuLCJpbnFzSI6Imh0dHBz0i8vZHVtbXlqc29uLmNvbS9pY29uL2VtaWx5cy8xMjgiLCJpYXQ0IjE3OTA5NTM0MDMsImV4cCI6MTc5MDk1NzA4M30.eSDpcYJTkWJrD4nZiA262J3f23sQinKn2_-Z14xCZM",
3   "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJlbWlseXMiLCJlbWFWb24uY29tIiwiaXN0IjoiJmZw1hbGUuLCJpbnFzSI6Imh0dHBz0i8vZHVtbXlqc29uLmNvbS9pY29uL2VtaWx5cy8xMjgiLCJpYXQ0IjE3OTA5NTM0MDMsImV4cCI6MTc5MDk1NzA4M30."
}
```

Open Folder Terminal Console Ready to go

Globals Vault Tools



Mario Maldonado's Workspace

Upgrade

UT PUT • GET GET • GET DELE • POST Log • GET Usuz • +

No Environment

MIA - Semana 13 - API > Usuario autenticado

GET https://dummyjson.com/user/me Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Auth type Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#)

Bearer Token

Token

200

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "firstName": "Emily",
4   "lastName": "Johnson",
5   "maidenName": "Smith",
6   "age": 29,
7   "gender": "female",
8   "email": "emily.johnson@x.dummyjson.com",
9   "phone": "+81 965-431-3024",
10  "username": "emilys",
11  "password": "emilyspass",
12  "birthDate": "1996-5-30",
13  "image": "https://dummyjson.com/icon/emilys/128",
14  "bloodGroup": "O-",
15  "height": 183.24
```

Open Folder Terminal Console Ready to go

Globals Vault Tools



The screenshot shows the Postman application interface. On the left, the 'COLLECTIONS' sidebar lists several API collections, with 'MIA - Semana 13 - API' expanded. The main workspace displays a GET request to 'https://dummyjson.com/user/me'. The 'Authorization' tab is active, showing 'No Auth' selected. The response body is visible, showing a 401 status code and a JSON message: 'message: Access Token is required'.

El Bearer Token es una credencial temporal utilizada para demostrar que una solicitud está autenticada. Después de iniciar sesión correctamente, la API devuelve un accessToken. Este token se envía posteriormente en el encabezado Authorization utilizando el formato Bearer <token>. Si el token es válido, la API permite acceder a recursos protegidos como /user/me. Sin el token, la API rechaza la solicitud y devuelve un código HTTP 401 Unauthorized.

Solicitud autenticada:

- Método: GET
- Endpoint: /user/me
- Autenticación: Bearer Token
- Resultado: información del usuario autenticado.



Universidad
Rafael Landívar

Identidad Jesuita en Guatemala

- HTTP: 200 OK

Solicitud sin autenticación:

- Método: GET
- Endpoint: /user/me
- Autenticación: ninguna
- Resultado: acceso rechazado.
- HTTP: 401 Unauthorized

Facultad de Ingeniería

Facultad de Informática y sistemas

Manejo e Implementación de Archivos

Viernes, 02 de octubre de 2026